NumPy 배열 연산 & Matplotlib

로봇 센서 데이터 처리 기초

cchyun@gmail.com

Part 1. NumPy — 왜 필요한가?

NumPy가 왜 필요한가? — 로봇과 실시간 연산

- **로봇은 '인지-판단-제어(Sense-Think-Act)' 루프를 끊임없이 반복**
  - 센서가 생성하는 **대량의 데이터**를 매우 짧은 시간 안에 계산해야 함
  - **Python 리스트의 한계**
    - 다양한 타입을 담을 수 있어 유연하지만, **메모리 오버헤드가 크고 속도가 느림**
    - 실시간 하드웨어 제어에 부적합
  - **NumPy의 강점 — 벡터화(Vectorization)**
    - C 언어 수준으로 최적화된 **N차원 배열(ndarray)** 제공
    - 대량 데이터를 **한 번에 계산** → 코드 간결 + 속도 향상

NumPy vs Python 리스트 — LiDAR 데이터 처리 비교

- **360개의 LiDAR 거리 데이터에 보정치를 더하는 경우**

  - **Python 리스트 방식 (느림)**

```
corrected = []
for d in distances:
    corrected.append(d + 0.1)  # 하나씩 순차 계산
```

  - **NumPy 방식 (빠름)**

```
import numpy as np
corrected = distances + 0.1  # 배열 전체에 한 번에 적용
```

  - 이러한 성능 차이는 **로봇의 제어 주기(Control Loop)를 안정적으로 유지**하는 데 결정적

Part 2. ndarray 구조 이해

ndarray의 구조 — Dimension, Shape, Dtype

- **NumPy의 핵심 객체 `ndarray`는 로봇의 물리적 상태를 숫자로 추상화**
  - **Dimension (차원)** — 데이터가 뻗어 나가는 방향
    - **1D (벡터):** 시간에 따른 IMU 가속도 값 (x, y, z)
    - **2D (행렬):** LiDAR 스캔 데이터 세트, 로봇 격자 지도(Grid Map)
  - **Shape (형태)** — 각 차원의 크기
    - 예: `(360,)` → 1도 단위로 측정된 LiDAR 거리 데이터 360개
  - **Dtype (데이터 타입)** — 로봇 데이터는 정밀도가 중요
    - 좌표, 속도, 전압 등 → 실수형 `float64`
    - 엔코더 틱, 센서 상태 코드 등 → 정수형 `int`

ndarray 구조 확인 실습

```python
import numpy as np

# 1D 배열: IMU 가속도 데이터 [x, y, z]
accel = np.array([0.02, -0.01, 9.81])
print(accel.shape)   # (3,)
print(accel.dtype)   # float64
print(accel.ndim)    # 1

# 2D 배열: LiDAR 360도 (x, y) 좌표
point_cloud = np.zeros((360, 2))
print(point_cloud.shape)   # (360, 2)
print(point_cloud.ndim)    # 2
```

- `shape`, `dtype`, `ndim` 속성을 수시로 확인하는 습관이 중요

Part 3. 자주 쓰이는 NumPy 연산

① 배열 생성 (Creation)

- **목적에 맞는 배열을 효율적으로 초기화**

  - `np.zeros()` — 모든 값이 0인 배열 생성
    - 로봇 메모리를 미리 확보하거나 데이터를 초기화할 때 사용

  - `np.ones()` — 모든 값이 1인 배열 생성
    - 보정 계수(Scale factor) 적용 시 유용

  - `np.linspace()` — 일정한 간격의 수열 생성
    - LiDAR 측정 각도 0°에서 360°까지 생성 시 필수

  - `np.random.random()` — 난수 생성
    - 센서의 측정 불확실성이나 **노이즈를 시뮬레이션**할 때 사용

① 배열 생성 — 코드 예시

```python
import numpy as np

# 0으로 초기화된 배열 (데이터 버퍼 사전 할당)
sensor_buffer = np.zeros(100)

# LiDAR 각도 배열: 0도 ~ 360도, 360개 포인트
angles = np.linspace(0, 2 * np.pi, 360)

# 시뮬레이션용 랜덤 거리 데이터 (0.5m ~ 5.0m)
distances = np.random.uniform(0.5, 5.0, 360)

print(angles.shape)     # (360,)
print(distances.shape)  # (360,)
```

② 슬라이싱과 인덱싱 (Slicing & Indexing)

- **로봇의 특정 영역에 있는 데이터만 골라낼 때 사용**
  - 기본 슬라이싱: `array[start:end]`
  - **예시: LiDAR 전방 데이터 추출**
    - `lidar_data[90:270]` → 전방 $90^\circ \sim 270^\circ$ 장애물 거리만 추출
  - **예시: 불리언 인덱싱으로 안전 거리 필터링**

```
lidar_data = np.array([2.5, 0.5, 1.8, 0.9, 3.0])

# 1.0m 미만의 장애물만 추출
obstacles = lidar_data[lidar_data < 1.0]
print(obstacles)  # [0.5 0.9]
```

③ 브로드캐스팅 (Broadcasting)

- **서로 다른 형태의 배열 간 연산을 자동으로 처리**
  - NumPy가 자동으로 값을 확장하여 연산 수행

```
import numpy as np

# 로봇 팔 3축 가속도 데이터
accel_data = np.array([0.02, -0.01, 9.81])      # shape: (3,)
calibration_offset = np.array([0.01, 0.01, 0.00])  # shape: (3,)

# 각 축에 자동으로 오프셋 적용
calibrated_accel = accel_data - calibration_offset
print(calibrated_accel)  # [0.01 -0.02  9.81]

# 스칼라 브로드캐스팅 (모든 값에 동일 연산)
scaled = accel_data * 100
print(scaled)  # [  2.  -1. 981.]
```

④ 수학 연산 (Math Operations)

- **센서 데이터 분석에 필수적인 통계·벡터 연산**
  - `mean()`, `std()` — 평균과 표준편차
    - 센서 값의 노이즈 특성을 분석
  - `min()`, `max()` — 최솟값·최댓값
    - LiDAR 데이터 중 가장 가까운 거리(`min`)를 찾아 **충돌 방지**
  - `np.argmin()`, `np.argmax()` — 최솟값·최댓값의 인덱스
    - 장애물 방향(각도) 탐색에 활용
  - `np.linalg.norm()` — 벡터의 크기(L2 Norm) 계산
    - IMU의 x, y, z 가속도 합성 → **전체 가속도 크기** 산출

수학 연산 — 코드 예시

```python
import numpy as np

lidar_data = np.array([4.2, 3.5, 1.1, 2.8, 0.8])

# 통계 분석
print(f"평균 거리: {np.mean(lidar_data):.2f}m")
print(f"표준편차: {np.std(lidar_data):.2f}m")
print(f"최근접 장애물: {np.min(lidar_data):.2f}m")

# 가장 가까운 장애물의 방향(인덱스)
min_idx = np.argmin(lidar_data)
print(f"장애물 방향 인덱스: {min_idx}")  # 4

# IMU 총 가속도 크기
accel = np.array([0.02, -0.01, 9.81])
total = np.linalg.norm(accel)
print(f"총 가속도: {total:.2f} m/s^2")
```

Part 4. 센서 데이터 처리 실습

센서 데이터 처리 예제 — LiDAR & IMU

```python
import numpy as np

# 1. LiDAR 데이터 처리: 각도 생성 및 거리 데이터 시뮬레이션
angles = np.linspace(0, 2*np.pi, 360)          # 0~360도 각도
distances = np.random.uniform(0.5, 5.0, 360)   # 랜덤 거리 데이터

# 특정 각도 범위(전방 30도)의 데이터만 추출 (슬라이싱)
front_data = distances[165:195]
if np.min(front_data) < 1.0:  # 가장 가까운 장애물이 1m 이내라면
    print("경고: 전방 장애물 감지! 정지 명령 수행")

# 2. IMU 가속도 데이터 처리
accel_data = np.array([0.02, -0.01, 9.81])        # [x, y, z] (m/s^2)
calibration_offset = np.array([0.01, 0.01, 0.00])

# 브로드캐스팅을 이용한 보정
calibrated_accel = accel_data - calibration_offset

# 총 가속도 크기(L2 Norm) 계산
total_accel = np.linalg.norm(calibrated_accel)
print(f"총 가속도: {total_accel:.2f} m/s^2")
```

센서 노이즈 시뮬레이션

- **현실의 센서는 완벽하지 않으며 항상 노이즈를 포함**
  - NumPy의 가우시안 분포로 노이즈를 쉽게 모사 가능

```
import numpy as np

angles = np.linspace(0, 2*np.pi, 360)
distances = np.random.uniform(0.5, 5.0, 360)

# 가우시안 노이즈 시뮬레이션 (5cm 수준)
noise_level = 0.05
noise = np.random.normal(0, noise_level, distances.shape)

# 노이즈가 섞인 실제 센서 데이터 생성
noisy_lidar = distances + noise

# 필터링 전후 비교
print(f"원본 평균 거리:       {np.mean(distances):.2f}m")
print(f"노이즈 포함 평균 거리: {np.mean(noisy_lidar):.2f}m")
```

NumPy 입문자가 자주 하는 실수

- **① 배열 연산에 `for` 문 사용하기**
  - NumPy 배열에 습관적으로 `for` 루프를 사용하면 성능 장점을 전혀 살릴 수 없음
  - 반드시 **벡터화 연산**을 우선 고려할 것

- **② Dimension/Shape 불일치**
  - `(3,)` 배열과 `(3, 1)` 배열 연산 시 의도치 않은 브로드캐스팅 또는 오류 발생
  - `shape` 속성을 수시로 확인하는 습관 필요

- **③ 데이터 타입(Dtype) 무시**
  - 정수형 배열에 실수값을 넣으면 소수점이 잘려 **데이터 왜곡** 발생
  - 센서 데이터는 되도록 `float64` 타입 유지

- **④ 얕은 복사(Shallow Copy) 실수**
  - `b = a`로 대입하면 `b` 수정 시 원본 `a`도 변경됨
  - 독립 복사본 필요 시 `a.copy()` 사용

Part 5. Matplotlib — 센서 데이터 시각화

Matplotlib의 구조 — 직관적 계층 이해

- **Matplotlib은 세 가지 계층으로 구성**

  - **Figure (캔버스)**
    - 그래프가 그려지는 **가장 바깥쪽의 전체 창** 또는 페이지
    - 하나 이상의 Axes를 포함할 수 있는 **최상위 컨테이너**

  - **Axes (좌표계/플롯)**
    - 실제로 **데이터가 그려지는 구역**
    - 제목, 축 레이블, 눈금 등을 포함
    - 우리가 보통 '그래프 하나'라고 부르는 단위

  - **Artist (그림 구성 요소)**
    - Figure에 보이는 **모든 것** (선, 텍스트, 점, 범례 등)
    - Axes에 데이터를 그리면 자동으로 Artist 객체 생성·관리

로봇 개발에서 자주 쓰이는 그래프 유형

- **① 시계열 라인 그래프 (Time-Series Plot)**
  - IMU, 엔코더 피드백 시각화
  - **PID 제어기 튜닝 시 오차 신호의 진동** 확인
  - 배터리 전압 변화 모니터링

- **② LiDAR 스캔 산점도 (Scatter Plot)**
  - 2D 공간에서 **장애물 탐지 및 경계 시각화**
  - 거리 데이터를 데카르트 좌표계 (x, y)로 변환 후 점으로 표시
  - **로봇 주변 환경 파악**에 활용

- **③ 히스토그램 (Histogram)**
  - 센서 데이터의 **노이즈 분포 분석**
  - 평균값 중심으로 데이터가 얼마나 퍼져 있는지 확인
  - 필터 알고리즘 설계의 기초 자료

센서 데이터 시각화 예제 코드

```python
import matplotlib.pyplot as plt
import numpy as np

# 시뮬레이션 데이터 생성
angles = np.linspace(0, 2 * np.pi, 360)
distances = 2 + 0.5 * np.random.randn(360)    # 노이즈 포함 LiDAR
x = distances * np.cos(angles)                # 극좌표 → 직교좌표
y = distances * np.sin(angles)

time = np.linspace(0, 10, 100)
imu_accel = np.sin(time) + 0.1 * np.random.randn(100)

# subplot으로 여러 데이터를 한 화면에 표시
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

ax1.scatter(x, y, s=5, c='red', label='LiDAR Scan')
ax1.set_title("LiDAR Obstacle Detection")
ax1.set_xlabel("X distance (m)")
ax1.set_ylabel("Y distance (m)")
ax1.grid(True)
ax1.legend()
ax1.set_aspect('equal')

ax2.plot(time, imu_accel, label='Accel Z-axis')
ax2.set_title("IMU Telemetry")
ax2.set_xlabel("Time (s)")
ax2.set_ylabel("Acceleration (m/s^2)")
ax2.grid(True)
ax2.legend()

plt.tight_layout()
plt.savefig("sensor_diagnostic.png")
plt.show()
```

그래프 꾸미기 및 관리

- **범례(Legend) 및 레이블**
  - 여러 센서(예: 왼쪽/오른쪽 IMU) 데이터를 동시에 그릴 때
  - **각 선이 무엇을 의미하는지 구분**하기 위해 필수

- **그리드(Grid)**
  - 로봇이 장애물과 얼마나 떨어져 있는지 **거리를 직관적으로 가늠**

- **파일 저장 (`plt.savefig()`)**
  - PNG 또는 PDF 형식으로 저장
  - 분석 결과를 **보고서 작성이나 사후 분석**에 활용

```
plt.tight_layout()
plt.savefig("sensor_diagnostic.png")  # PNG 또는 PDF
plt.show()
```

Matplotlib 입문자가 자주 하는 실수

- **① NumPy 미사용**
  - 대용량 데이터에 Python 리스트를 사용하면 시각화 속도가 매우 느림
  - 반드시 **벡터화된 NumPy 연산** 후 플로팅

- **② 실시간 루프 내 Figure 생성**
  - `while` 루프 안에서 매번 `plt.figure()` 호출 → **메모리 부족으로 시스템 중단**
  - Figure는 **루프 밖에서 한 번만 생성**하고 데이터만 업데이트

- **③ 축 범위(Limit) 미설정**
  - 로봇 이동에 따라 축 범위가 자동 변경 → 상대적 변화 관찰 어려움
  - `ax.set_xlim()`, `ax.set_ylim()`으로 **고정된 시야 확보**

- **④ 차단형(Blocking) 플로팅**
  - `plt.show()`는 기본적으로 **프로그램 실행을 멈춤**
  - 제어 루프와 시각화를 동시에 하려면 `plt.ion()` (대화형 모드) 활용

실습 체크리스트

- **복습 시 다음 항목들을 제대로 수행했는지 점검하세요**
  - [ ] **가상 환경(venv)** 을 활성화하고 `numpy`, `matplotlib` 패키지가 정상 설치되었는가?
  - [ ] 센서 데이터 처리에 리스트 대신 **NumPy 배열**을 사용하여 계산 속도를 높였는가?
  - [ ] LiDAR 데이터 좌표 변환 시 `for` 루프 대신 **NumPy 벡터화 연산**을 적용했는가?
  - [ ] 로봇의 상태(전압, 전류, PID 오차)를 시각화하여 **디버깅 도구**로 활용하고 있는가?
  - [ ] 그래프에 **제목(Title)과 축 이름(Label)** 을 작성하여 다른 개발자가 이해할 수 있게 했는가?